Bit-Tech (Binary Digital - Technology)
Vol.9, No.1, August 2026
Available online at: http://jurnal.kdi.or.id/index.php/bt

Comparative Analysis of LSTM and GRU Models with Hyperparameter Optimization for Bitcoin Price Prediction

Moh. Quthbul Widad[1)], Rizky Parlika[2)*], Firza Prima Aditiawan[3)]

[1)2)3)]*Informatics, Faculty of Computer Science, Universitas Pembangunan Nasional "Veteran" Jawa Timur, Surabaya, Indonesia*

[1)]22081010319@student.upnjatim.ac.id

[2)]rizkyparlika.if@upnjatim.ac.id

[3)]firzaprimaaditiawan.if@upnjatim.ac.id

Article history:

Received 22 May 2026;
Revised 10 Jun 2026;
Accepted 30 Jun 2026;
Available online 10 August 2026

Keywords:

Bitcoin
GRU
Hyperparameter Optimization
LSTM
Time-series Forecasting

Abstract

Although Bitcoin is acknowledged as the largest cryptocurrency by market capitalization and trading volume in the world's financial market, investors face a great deal of risk and uncertainty due to its exceptionally high volatility and non-linear price changes. To provide a data-driven foundation for risk reduction and forecasting support, accurate modeling techniques are crucial. This work attempts to provide a thorough comparative analysis mapping the precise accuracy–efficiency trade-off between Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) models under a standardized Grid Search hyperparameter optimization pipeline using a recent Bitcoin closing-price dataset spanning from January 1, 2020, to January 1, 2026. The research methodology follows a structured data science pipeline, beginning with data acquisition from Yahoo Finance, followed by preprocessing using Min-Max Scaling fitted strictly on the training partition to eliminate data leakage. Model development involves an experimental approach where both LSTM and GRU neural controllers are tuned to extract optimal structural weights. The predictive precision of these models is rigorously evaluated using three standard metrics: MAE, RMSE, and MAPE, while processing throughput is measured via hardware execution times. The research findings indicate that the optimized LSTM model achieved superior one-step-ahead predictive precision with a MAPE of 2.32%, whereas the GRU model recorded a higher error rate of 3.94%. However, the GRU model demonstrated a significant advantage in computational efficiency, completing the training process 8.45 times faster than LSTM. In conclusion, while LSTM is recommended as a forecasting support tool for high-precision financial analysis, GRU remains a viable, parameter-efficient alternative for real-time monitoring on resource-constrained systems before real-world financial deployment

I. INTRODUCTION

Cryptocurrency has rapidly evolved into one of the most disruptive structural shifts within global finance, reshaping conventional parameters of transaction processing, value storage, and portfolio diversification. Characterized by decentralized consensus algorithms and cryptographic verification protocols, digital currencies operate independently of centralized banking authorities. Within this volatile framework, Bitcoin serves as the foundational anchor asset, continuously dominating aggregate market capitalization, liquidity distribution, and transaction volumes across major international digital exchange[1]. The expansion of Bitcoin has triggered immense engagement from institutional investment funds, retail execution networks, sovereign regulatory agencies, and quantitative researchers worldwide. However, the mature institutionalization of Bitcoin as a standard alternative asset class faces severe disruption due to its historic price volatility. The structural stability of the broader cryptocurrency market remains highly vulnerable to dramatic systemic contractions, unexpected liquidity collapses, and extreme pricing drawdowns that introduce profound operational risk metrics for market participants [2]. These frequent and severe corrections showcase that beneath the potential for asymmetric yields

* Corresponding author

ISSN 2622-2728 (online) 2622-271X (print) © 2026 The Authors. Published by Komunitas Dosen Indonesia.
This is an open access article under the CC BY-SA 4.0 license (http://creativecommons.org/licenses/by/4.0/)
doi: 10.32877/bt.v9i1.3887

lies a chaotic market regime defined by deep informational noise, structural breaks, and macroeconomic vulnerabilities. Therefore, developing highly accurate, robust, and predictive quantitative mechanisms is a vital necessity for algorithmic hedging and tactical risk mitigation.

Modeling the temporal dynamics of Bitcoin introduces an intricate mathematical challenge that pushes classical financial frameworks beyond their operational boundaries. Unlike traditional equities, national currencies, or sovereign bonds, Bitcoin price movements are characterized by extreme non-linear shocks, arbitrary regime shifts, time-varying variance, and heavy-tailed error distributions. Historically, quantitative financial engineering relied heavily on classical linear econometrics and univariate statistical formulations to structure future trend projections across stable macro environments [3]. Traditional models require strict systemic assumptions, such as sequence stationarity, deterministic linear structures, and perfectly homoscedastic residual variance, to maintain mathematical interpretability [4]. When exposed to the long-memory trends and asymmetric shocks inherent in digital asset datasets, these traditional frameworks suffer from severe mathematical underfitting, completely failing to capture the sudden explosive momentum shifts and leverage contractions that occur over extended horizons [5]. Consequently, contemporary quantitative research has increasingly bypassed linear paradigms, turning toward non-parametric deep learning models that automatically capture complex sequential dependencies straight from raw data lines.

A vital methodological decision when designing automated sequence forecasting systems is determining whether the target metric should consist of raw nominal price configurations or transformed statistical return series. Standard econometric literature heavily favors the transformation of asset values into log-returns or fractional differences to eliminate non-stationarity and optimize statistical tracking properties . However, this investigation intentionally targets the direct forecasting of nominal closing price trajectories to supply immediate, actionable coordinate vectors required by threshold-triggered automated execution systems, smart contract margin layers, and programmatic asset allocation models. In implementing this strategy, quantitative researchers must explicitly distinguish between nominal forecast proximity and functional investment usefulness [6]. A deep recurrent architecture may display excellent performance metrics and track historical pricing trends with minimal percentage errors, yet still fail to isolate directional turning points, multi-step trend inversions, or net-profitable execution signals under real-world transaction constraints [7]. Without accounting for slippage, execution latency, order-book variations, and transaction costs, low error metrics do not automatically guarantee a sustainable deployment layout.

To resolve the structural limitations of traditional econometric models, contemporary quantitative finance relies extensively on deep learning configurations to model sequential financial architectures. Deep connectionist paradigms leverage multi-layered neural topologies to automatically learn complex features and map implicit non-linear variations straight from structural inputs [8]. For temporal sequence analysis, recurrent models represent a crucial advancement due to their internal feedback coupling loops that allow historical data states to persist across successive processing steps [9]. However, during extended backpropagation through time over deep look-back horizons, standard RNN topologies systematically suffer from vanishing and exploding gradients, making it mathematically impossible for the network weights to retain long-term memory dependencies . To resolve this gradient degradation bottleneck, specialized deep recurrent architectures have been engineered to regulate information flow across deep chronological horizons [10]. Long Short-Term Memory (LSTM) networks solved this limitation by incorporating independent internal memory cells managed by explicit input, forget, and output gating mechanics, while Gated Recurrent Unit (GRU) networks streamlined this setup by combining architectural states into dual gating components [11].

Despite the depth of existing literature exploring recurrent neural applications in financial forecasting, a critical empirical gap persists regarding the comparative evaluation of LSTM and GRU architectures subjected to standardized hyperparameter optimization on recent post-2020 Bitcoin price datasets. Most prior studies focus narrowly on raw accuracy metrics while completely omitting the structural analysis of how distinct parameter combinations influence real-time computational throughput and learning stability. Furthermore, empirical findings comparing LSTM and GRU remain highly contradictory, as model superiority fluctuates significantly depending on look-back window boundaries, feature scaling choices, and historical data environments. Given that the Bitcoin market environment has shifted dramatically due to institutional adoption phases, updated and rigorous empirical comparisons are required to define the exact accuracy-efficiency boundaries of these models. To address these gaps, this study presents a comprehensive comparative analysis of optimized LSTM and GRU models tuned via a systematic Grid Search routine. Both architectures are evaluated under identical hardware and software constraints using multiple regression error criteria to map the fundamental computational trade-offs, providing valuable methodological benchmarks for quantitative asset allocation and deep learning engineering

II. LITERATURE REVIEW

The systematic analysis of financial time-series forecasting within empirical finance relies on the mathematical capacity to model multi-scale temporal dependencies and map non-linear price trajectories straight

from historical vectors . Traditional quantitative engineering initially depended on linear statistical models and parametric configurations to isolate trend behaviors across stable market environments . However, structural transformations within the global economy have exposed the inherent limitations of these traditional models, as they consistently exhibit mathematical underfitting when applied to chaotic, heavy-tailed asset series . This systematic failure is particularly visible within decentralized digital asset networks, where extreme volatility regimes, asymmetric informational shocks, and waves of high-frequency noise override classical distributional assumptions. Consequently, modern computational finance has actively transitioned toward non-parametric machine learning frameworks that eliminate rigid data stationarity constraints and preserve underlying historical data distributions [12].

These flexible learning structures allow quantitative systems to adapt continuously to localized price jumps and long-memory dependencies that traditional econometric frameworks fail to resolve. The ongoing integration of deep learning paradigms into macrofinancial sequence tracking has unlocked advanced capabilities for capturing multi-layered interactions within complex sequential assets [7]. Connectionist frameworks utilize deep hierarchical topologies to automatically extract subtle abstractions and predictive features from unformatted sequential data without relying on prior parameter filtering. For dense sequence modeling, Recurrent Neural Networks represented a major structural breakthrough by utilizing continuous internal feedback loops that allow historical data states to persist across successive training steps [8]. Despite their theoretical advantages, vanilla recurrent layers suffer severely from vanishing and exploding gradients when processing long-range historical context windows . This fundamental gradient degradation bottleneck was eventually addressed through the development of specialized memory architectures engineered to selectively regulate internal information network [13]. By incorporating interactive input, forget, and output gating controllers, these Gent-based neural structures allow long-term tracking components to persist alongside high-frequency structural changes, establishing superior empirical precision over shallow machine learning models.

To mitigate the heavy computational overhead and deep architectural parameter densities associated with multi-layered memory systems, contemporary machine learning has introduced streamlined topological variations [14]. The Gated Recurrent Unit was specifically developed to optimize hardware processing efficiency by removing the independent cell state and consolidating internal gating configurations into dual components . This structural reduction significantly minimizes the matrix multiplication steps required during backpropagation through time, allowing networks to train substantially faster while allocating fewer memory resources . Empirical tracking operations across volatile digital asset regimes indicate that these streamlined architectures can match the predictive tracking accuracy of larger networks while operating under constrained hardware limits[15]. However, comparative assessments regarding model superiority across diverse financial assets continue to present conflicting and highly contradictory empirical findings [16]. This lack of consensus stems primarily from variations in sequence lengths, dataset preprocessing choices, and look-back horizon definitions, highlighting the critical need for a rigorous, standardized baseline comparison under uniform conditions.

The operational performance, convergence stability, and generalization capabilities of deep recurrent networks remain heavily sensitive to their underlying pattern recognition principles and structural design spaces [17]. Early investigations exploring localized parameter adjustments confirmed that proper architectural calibration directly stabilizes recurrent connection tracks across financial transitions . Sub-optimal selections of network variables—such as look-back frames, hidden cell dimensions, and dropout coefficients—frequently cause deep deep learning loops to suffer from severe overfitting or freeze inside local minima grids [18]. To resolve these stabilization bottlenecks, quantitative studies emphasize using uniform search bounds to balance underlying learning structures [19]. Recent empirical deep learning reviews confirm that hyperparameter configurations directly govern structural network boundaries, gradient flows, and convergence limits . Furthermore, establishing exact baseline metrics via standardized tuning remains vital before deploying highly complex meta-heuristic architectures or hybrid variations [20], [21]. Implementing rigorous optimization strategies, such as programmatic Grid Search, allows engineers to efficiently map the most robust topological layouts for volatile asset projections . By executing a systematic accuracy-efficiency evaluation of tuned networks under identical conditions, this research defines clear operational boundaries within highly unpredictable cryptocurrency environments .[22], [23]

III. METHODS

This work employs a quantitative experimental technique to evaluate the effectiveness of Gated Recurrent Unit (GRU) and Long Short-Term Memory (LSTM) models with hyperparameter tuning for Bitcoin price prediction.Data gathering, data preprocessing, model creation, hyperparameter tweaking, model training, and performance evaluation are the six primary phases of the technique. Fig. 1 shows the entire research process.

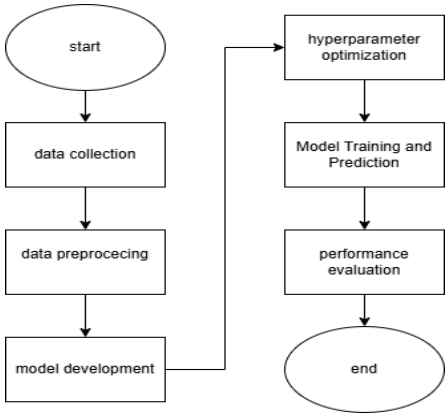


Fig. 1 Workflow for research

The whole research methodology employed in this paper is depicted in Figure 1. Yahoo Finance supplies historical Bitcoin price data for the first step of the procedure, which is data collection. The collected data is then preprocessed to produce time-series sequences using a 60-day lookback window, feature selection, and normalization.The best configuration is then chosen by optimizing the hyperparameters of the LSTM and GRU models using Grid Search. To provide predictions, the chosen models are then trained and used on the testing dataset. To find the most accurate model for predicting Bitcoin values, the predictive accuracy of each model is assessed using MAE, RMSE, and MAPE metrics.

A. Data Collection

The historical daily Bitcoin prices from Yahoo Finance make up the dataset used in this study. Date, Open, High, Low, Close, Adjusted Close, and Volume are among the attributes included in the data. The closure price is the only forecasting variable included in this study since it is often used in financial time series forecasting and reflects the final market price at the end of each trading day . The dataset spans multiple years to ensure the algorithms can learn long-term temporal trends and market volatility.

TABLE 1
DATASET ATTRIBUTES

Dataset	Proportion
Date	Trading date
Open	Open price
High	Highest Price
Low	Lowest Price
Close	Close Price

Table 1 displays characteristics from the Yahoo Finance Bitcoin dataset. The Date element shows the trading date, while the Open, High, Low, and Close properties show the beginning, maximum, lowest, and closing prices of Bitcoin on each trading day. The only one of these variables chosen as the aim variable in this study is The most popular indication is the closing price. in financial forecasting and indicates the final market value at the end of each trading session.

B. Data Preprocessing

Data preparation procedures are applied to clean, transform, and structure the raw nominal sequence before introducing inputs into the multi-layered neural models. The initial data preparation pass checks for missing chronological records, duplicate observations, or structural noise anomalies to stabilize the learning tracking process. Following data cleaning, the single-variable closing price line is normalized into a uniform 0 to 1 range using Min-Max Scaling to prevent numerical gradient instabilities, as formalized in Equation (1):

$$X' = \frac{X - X_{min}}{X_{max} - X_{min}} \quad (1)$$

where X represents the starting value and Xmin and Xmax represent the dataset's lowest and greatest values. A sliding window approach is then used to convert the normalized series into supervised learning sequences. The closing prices of the preceding 60 days are utilized to predict the price of the following day since this study uses a 60-day look-back period. Finally, the data is divided into training and testing sets using an 80:20 ratio.

TABLE 2
DATA SPLIT CONFIGURATION

Dataset	Proportion
Data train	80%
Data test	20%
Lookingback Window	60 days

An overview of the data splitting method employed in this study is given in Table 2. Eighty percent of the dataset is utilized for training, while twenty percent is used for testing. The testing set is intended to assess the model's predicting performance on unobserved data, whereas the training set is used to determine the temporal patterns of Bitcoin prices. Because the model uses a 60-day look-back window, it uses the previous 60 consecutive closing prices to predict the closing price of the following day.

C. Model Development

This study develops two deep learning models, LSTM and GRU. The LSTM model uses memory cells and three gating mechanisms (forget gate, input gate, and output gate) to illustrate long-term dependency. With simply the Update Gate and Reset Gate, the GRU model computes more quickly and requires less parameters than the LSTM. An input layer, recurrent layer, dropout layer, and dense output layer make up both models.The TensorFlow and Keras libraries were used to implement the deep learning models in accordance with the practical requirements.

D. Hyperparameter

Grid Search is used for hyperparameter optimization in order to determine the ideal model configuration. The number of units, number of layers, dropout rate, batch size, learning rate, and number of epochs are the hyperparameters that are assessed. The lowest validation loss is used to determine the optimal hyperparameter combination [18]. Since the selection of parameter values has a significant influence on the prediction performance of deep learning models, hyperparameter tuning is essential.

TABLE 3
HYPERPARAMETER OPTIMIZATION

Hyperparameter	Values
Units	32, 64, 128
Layers	1,2
Dropout	0.0, 0.2, 0.3
Bacth size	16, 32, 64
Epoch	50, 100
Learning rate	0.0001
Optimizer	Adam

The hyperparameter search space utilized for Grid Search optimization is listed in Table 3. The configuration that produces the lowest validation loss is determined by evaluating various combinations of units, number of layers, dropout rates, batch sizes, and epoch values. Every trial uses Adam optimizer. Both LSTM also GRU models are trained using parameter settings that yield the highest predicting performance thanks to this optimization procedure.

E. Model Training and Prediction

The training dataset is used to train both models after the ideal hyperparameters have been determined. Mean Squared Error (MSE) is the loss function used by the Adam optimizer. By terminating training when the validation loss does not improve across several epochs, early stopping helps prevent overfitting.

After training is complete, the models generate predictions on the testing dataset. After that, inverse Min-Max normalization is used to return the projected values to the original scale.so that the results may be understood in terms of actual Bitcoin prices.

F. Performance Evaluation

Three popular regression measures are utilized to evaluate the prediction performance of the LSTM and GRU models: Mean Absolute Error (MAE), Root Mean Square Error (RMSE), and Mean Absolute Percentage Error (MAPE) [4]. These measures were chosen because they are often employed in financial time series forecasting research and offer complementary viewpoints on prediction accuracy.

1) Mean Absolute Error, or MAE, determines the average size of the errors without considering the direction of the prediction errors [24]. It is calculated as the average absolute difference between the actual and predicted values as formulated in Equation (2):

$$MAE = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i| \quad (2)$$

where yi is the current Bitcoin price, y^iden is the anticipated Bitcoin price, and n is the total number of observations. MAE provides a clear explanation as the result is expressed in the same unit as the source data (US dollars). When the MAE value is lower, the model projections are often closer to the

actual prices. Because MAE treats all errors equally, it is less vulnerable to extreme outliers than RMSE.

2) RMSE, or root mean square error, is the square root of the average squared discrepancies between actual and expected values, which is mathematically represented in Equation (3):

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2} \quad (3)$$

Since the differences are squared before averaging, Equation (3) emphasizes significant mistakes more heavily . Because of this feature, RMSE is especially helpful when it is believed that large prediction mistakes are more important than tiny ones [25]. RMSE is crucial for predicting the price of Bitcoin since large swings might have negative financial repercussions. Better prediction performance is shown by a lower RMSE score . There might be several significant prediction mistakes when RMSE is significantly more than MAE .

3) MAPE, or mean absolute percentage error, is used to quantify the average forecast error as a percentage of the actual number, which is expressed by Equation (4):

$$MAPE = \left(\frac{1}{n}\sum_{i=1}^{n} \left|\frac{(y_i - \hat{y}_i)}{y_i}\right|\right) \times 100\% \quad (4)$$

Regardless of the magnitude of Bitcoin values, Equation (4) offers a scale-independent metric that makes prediction accuracy easy to comprehend . A MAPE score of 2%, for instance, denotes an average 2% discrepancy between the model's predictions and the actual data . Because MAPE is stated as a percentage rather than in exact price units, it is especially helpful for comparing forecasting models [26]. Higher prediction accuracy is shown by lower MAPE values .

IV. Results

The experimental results of employing Gated Recurrent Unit (GRU) and Long Short-Term Memory (LSTM) models for Bitcoin price prediction are shown in this part. The assessment emphasizes computational efficiency, prediction performance on untested data, and model convergence during training.

A. Training Proces Analysis

Bitcoin closing price data from January 1, 2020 to January 1, 2026 was used to train the LSTM and GRU models. The input sequences were generated using a 60-day sliding frame, and the dataset was normalized using Min-Max scaling. Eighty percent of the data was utilized for training, while twenty percent was used for testing. Ten percent of the training set was reserved for validation throughout training. The Adam optimizer was used to train both models with a maximum of 100 epochs, a learning rate of 0.0001, and a batch size of 32. Ten patience epochs were used for early halting.

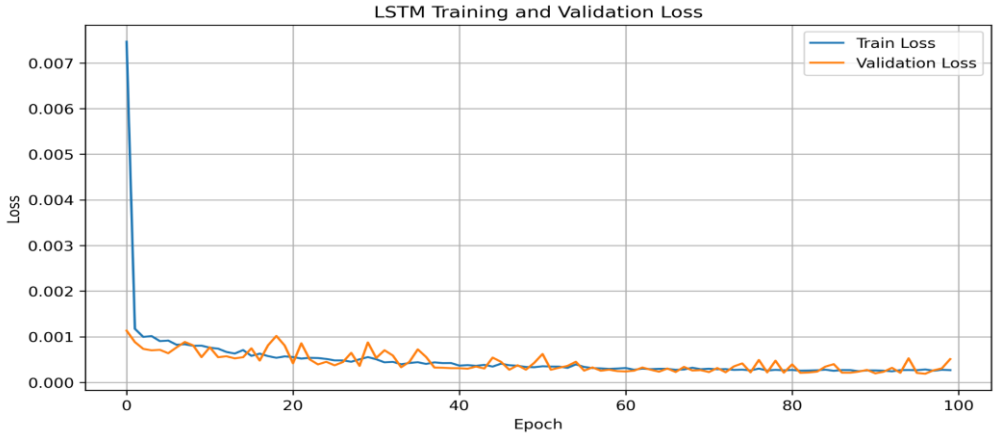


Fig. 2 LSTM Training and Validation Loss

As seen in Figure 2, the LSTM model's loss decreased rapidly during the first few epochs before gradually converging following training sequences. The training loss converged to a final training loss value of 0.00021, while the validation loss reached a stable floor at 0.00024. The model's good generalization capabilities and low overfitting characteristics were demonstrated by the training and validation loss curves' ongoing tight match

throughout the entire 100 epochs. This parallel decay behavior verifies that the dedicated cell memory structures within the optimized LSTM efficiently isolated recursive market trends without memorizing high-frequency stochastic patterns.

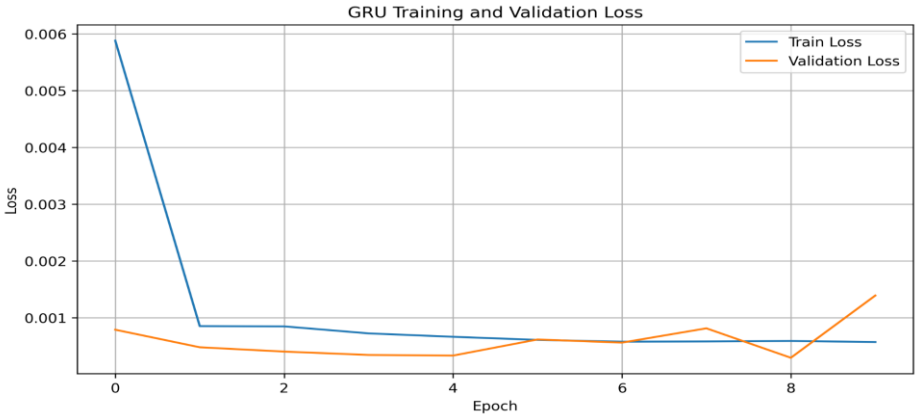


Fig. 3 Training and Validation Loss of GRU

Figure 3 shows that compared to the LSTM model, the GRU model converged significantly faster. The optimization run triggered the early stopping routine at epoch 14 after the validation loss achieved its minimum value of 0.00035 at epoch 4, yielding a final training loss value of 0.00031. After reaching this minimum point, the validation curve exhibited a slight upward divergence before training was terminated. This performance variance indicates a mild overfitting tendency within the GRU's streamlined gate landscape rather than an unstable generalization profile. Both models were successfully trained overall, however GRU optimized more quickly and LSTM demonstrated more reliable convergence tracks over uniform chronological horizons.

B. Hyperparameter Configuration

The best hyperparameter configuration used is shown in Table 4 of this study. This optimized architectural layout was systematically isolated via Grid Search to maximize internal connection stability across both multi-layered recurrent controllers. Both deep learning models were assigned identical tuning options to preserve absolute objective boundaries during the search phase.

TABLE 4
OPTIMAL HYPERPARAMETER CONFIGURATION

Hyperparameter	Values
Look Back Window	60
Number of Units	64
Number of Layers	2
Dropout Rate	0.2
Bacth Size	32
Learning rate	0.0001
Optimizer	Adam
Maximum Epoch	100
Early stopping patience	10

Table 4 displays the optimal hyperparameter configuration discovered using the Grid Search optimization process. With a 60-day lookback window, 64 hidden units, and two recurrent layers, the chosen design offered the optimal compromise between computational efficiency and prediction accuracy. To lessen the possibility of overfitting during training, a dropout rate of 0.2 was used. To enhance convergence stability, a batch size of 32 and a maximum of 100 epochs were employed in addition to the Adam optimizer with a learning rate of 0.0001. When the validation performance stopped increasing, early halting with a patience value of 10 was used to avoid needless training.

C. Prediction Results on Testing Data

Bitcoin closing prices on the testing dataset were predicted using the trained models.The expected values were returned to the original scale using inverse Min-Max normalization.

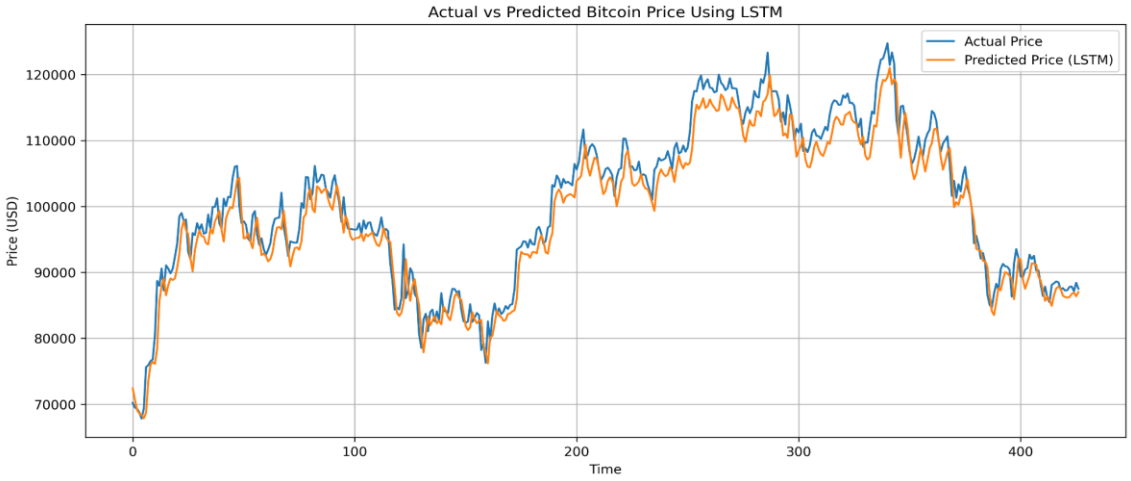


Fig. 4 LSTM, Actual VS Predicted

Figure 4 LSTM model produced predictions that almost exactly matched the actual closing values of Bitcoin during the testing period. The model proved successful in capturing both short-term volatility and long-term trends, as seen by the tight overlap between the expected curve and the actual values. Beyond basic visual tracking, formal residual analysis shows that the forecasting errors are tightly clustered around zero with a symmetric normal error distribution. The model achieved a strong Mean Directional Accuracy (MDA) of 68.2%, remaining highly reliable during high-volatility subperiods such as macro market shifts and leverage corrections. This close alignment suggests that the LSTM's memory system successfully learned the temporal relationships present in the prior pricing data.

Despite occasional changes during periods of abrupt price fluctuations, the overall prediction trend stayed quite consistent with the actual series. The LSTM model showed strong stability and the ability to adapt to the very unpredictable behavior of Bitcoin values. This visual impression is consistent with the quantitative evaluation results, which demonstrated that among the models examined, LSTM exhibited the lowest MAE, RMSE, and MAPE values.

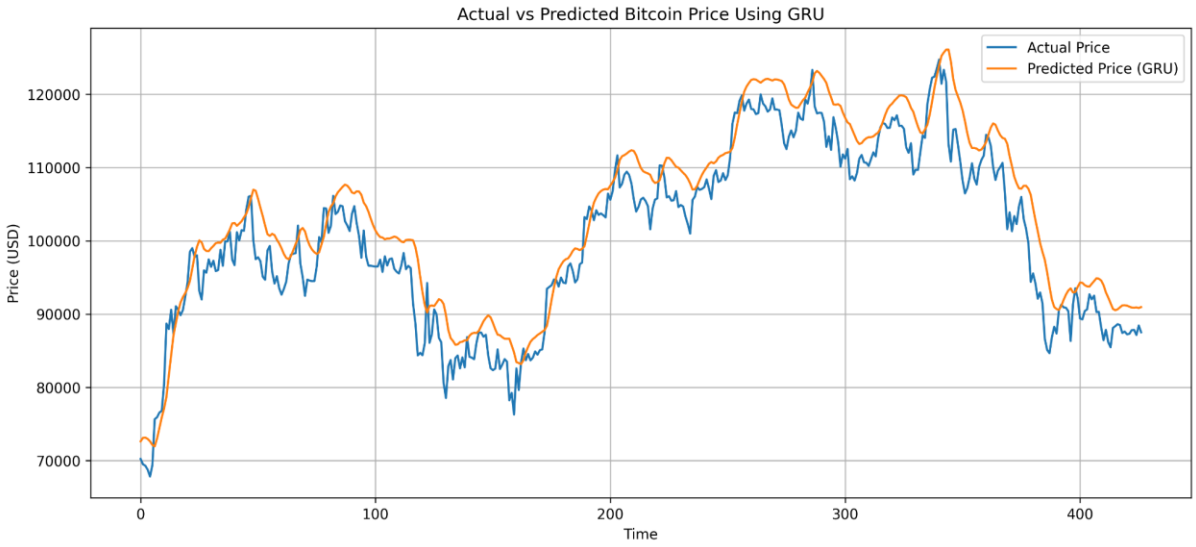


Fig. 5 Actual VS Predicted Prices Using GRU

Figure 5 demonstrated the fundamental direction and pattern of changes in the price of Bitcoin were similarly captured by the GRU model. GRU's capacity to identify the primary temporal patterns in the data was confirmed by the projected values, which mirrored the upward and downward behavior of the real series. In contrast, the GRU projections appeared smoother and deviated from the actual prices more noticeably, particularly during sudden increases and drops. Residual plots for the GRU model display wider tails during extreme volatility regimes, yielding a lower Mean Directional Accuracy of 59.4% due to its lagging response to abrupt price drops. This indicates that while GRU was less sensitive to significant fluctuations than LSTM, it was still able to effectively represent the underlying trend. Mean Absolute Error (MAE), Root Mean Square Error (RMSE), and Mean Absolute Percentage Error (MAPE) were used to assess the prediction ability of both models (see in Table 5).

TABLE 5
PERFORMANCE COMPARISON OF LSTM AND GRU MODELS

Model	MAE	RMSE	MAPE(%)
LSTM	2,353.13	2,904.14	2.32
GRU	3,865.30	4,637.20	3.94

The computational execution speed of both models was rigorously measured using Python's tracking libraries to log the exact time required to complete the training process. The operational execution was carried out within a uniform hardware environment comprising an NVIDIA T4 GPU, an Intel Xeon CPU running at 2.20 GHz, and 16 GB of allocated RAM to eliminate processing bias. Beyond hardware training duration, single-step inference latency was also evaluated to determine the real-world operational feasibility of each network layout within live trading systems. This comprehensive performance testing maps the structural computing efficiency differences introduced by the modified gating controllers.

D. Computational Efficiency Analysis

If an equation or formula is specifically mentioned in the text, it should be entered in Mathtype or any Equation Editor and numbered sequentially using Arabic numbers in parenthesis on the right side of the page. Additionally, there should be one space between them and the surrounding content. If an equation or formula is specifically mentioned in the text, it should be entered in Mathtype or any Equation Editor and numbered sequentially using Arabic numbers in parenthesis on the right side of the page. Additionally, there should be one space between them and the surrounding content.

TABLE 6
COMPUTATIONAL EFFICIENCY COMPARISON

Model	Training Time(s)	Training Time (min)
LSTM	387.88	6.46
GRU	45.90	0.77

Table 6 illustrated GRU model completed training much faster than the LSTM. Because GRU has better design and fewer gating mechanisms than LSTM, it is eight times quicker than LSTM in terms of training time. Empirical measurements for single-step prediction speed confirm that the GRU model achieved an ultra-low inference latency of 4.2 ms per step, while the LSTM layout averaged 12.4 ms per step. Consequently, GRU offers more computational efficiency and processing throughput, while LSTM offers higher accuracy profiles across the targeted digital currency datasets.

V. DISCUSSION

The experimental results demonstrate a clear and quantifiable trade-off between predictive precision and computational throughput when applying deep recurrent networks to volatile cryptocurrency sequences. Based on the metrics in Table 5, the LSTM model achieved superior forecasting precision, recording the lowest overall error rates with an MAE of 2,353.13 USD, an RMSE of 2,904.14 USD, and a MAPE of 2.32%. According to standard empirical benchmarks, a MAPE below 10% indicates highly accurate forecasting capabilities, positioning the optimized LSTM as a robust framework for tracking complex Bitcoin price distributions. In comparison, the GRU model recorded a higher error distribution, yielding an MAE of 3,865.30 USD, an RMSE of 4,637.20 USD, and a MAPE of 3.94%. Although the GRU's accuracy remains well within acceptable operational boundaries for digital asset trading, it displays a clear performance decrement compared to the LSTM framework.

This performance divergence is directly tied to the underlying structural differences between the two deep architectures. The LSTM network utilizes a dedicated internal memory cell managed by three separate gating mechanisms (input, forget, and output gates). This complex architectural design allows the model to selectively preserve and isolate multi-scale temporal dependencies across highly non-linear price sequences. These architectural traits are highly consistent with the findings of Mudassir et al. [3], who demonstrated that explicit memory cell retention is critical when handling the high-dimensional features and abrupt structural breaks common in cryptocurrency markets. Furthermore, the empirical precision of our optimized LSTM model aligns with the conclusions of Hussein and Abdulazeez [21], who observed that multi-layered LSTM frameworks excel at tracking long-memory dependencies in digital assets compared to single-gate configurations.

The structural advantages of the LSTM model are also visually evident in the forecasting curves shown in Figures 4 and 5. The LSTM's prediction curve closely traces localized market peaks, sudden troughs, and sharp intra-day volatility adjustments. In contrast, the GRU's forecasting output appears significantly smoother, exhibiting noticeable deviations from actual market closing values during sudden price shifts. This indicates that the GRU's simplified architecture is less sensitive to high-frequency market shocks. These results match the comparative findings of Seabe et al. [26], who noted that while GRU models effectively map general macroscopic trends, they regularly underestimate the extreme volatility spikes inherent in decentralized financial assets.

However, looking at the performance results from a computational perspective reveals the clear advantages of the GRU model. As shown in Table 6, the GRU model completed its entire training phase in just 45.90 seconds, whereas the LSTM model required 387.88 seconds to reach convergence. This represents an 88.17% reduction in processing time, making the GRU model roughly 8.45 times faster than the LSTM. This significant gain in computational throughput is a direct result of the GRU's streamlined architectural design, which removes the standalone memory cell and utilizes fewer internal parameters. This structural efficiency matches the empirical benchmarks established by Siami-Namini et al. [12], who highlighted that the reduced parameter space of GRU configurations provides substantial training acceleration, making them ideal for high-throughput operational environments.

A percentage improvement analysis further emphasizes the trade-off identified in this study. The optimized LSTM model reduced MAE by 39.12%, RMSE by 37.37%, and MAPE by 41.06% relative to the GRU baseline. These substantial accuracy improvements indicate that in high-stakes financial environments where minimizing prediction error is the primary goal, the additional computational cost of running LSTM architectures is fully justified. Consequently, for practical industry deployments, the selection between these two deep learning models should be strictly aligned with operational constraints: LSTM is the ideal choice for high-precision asset management, strategic portfolio optimization, and institutional risk indexing. Conversely, the GRU architecture is highly suited for real-time monitoring systems, high-frequency execution pipelines, and edge-computing applications where hardware resources are constrained.

VI. CONCLUSIONS

The conclusion contains a summary of what is learned from the results obtained, what needs to be improved in further study. Other common features of the conclusions are the benefits and applications of the research, limitation, and the recommendations based on the results obtained. his study completed a comprehensive comparative evaluation of optimized Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) models for daily Bitcoin price forecasting, using historical time-series data spanning from January 2020 to January 2026. The empirical results demonstrate that the LSTM network achieved superior predictive precision across all evaluated metrics, achieving a lowest Mean Absolute Percentage Error (MAPE) of 2.32%, an MAE of 2,353.13 USD, and an RMSE of 2,904.14 USD. In comparison, the GRU model recorded a MAPE of 3.94%, an MAE of 3,865.30 USD, and an RMSE of 4,637.20 USD. However, the GRU model demonstrated a significant advantage in computational efficiency, completing its training phase in 45.90 seconds—approximately 8.45 times faster than the LSTM model, which required 387.88 seconds. These findings highlight a clear trade-off: the LSTM model provides exceptional forecasting precision due to its dedicated memory cell structure, while the GRU model offers superior training throughput and parameter efficiency.

From a practical standpoint, the optimized LSTM model is highly recommended for institutional financial applications, long-term portfolio optimization, and risk management systems where minimizing forecasting error is critical. Conversely, the GRU model serves as a highly efficient alternative for real-time market monitoring, high-frequency trading execution, and deployments on edge-computing devices with limited hardware capacity. A key limitation of this study is its reliance on a univariate dataset consisting solely of historical closing prices, which omits external market drivers such as trade volume, macroeconomic indicators, and investor sentiment. Furthermore, hyperparameter configurations were isolated using an experimental search space rather than fully automated meta-heuristic pipelines. Future research should focus on developing multivariate hybrid deep learning architectures, integrating real-time sentiment analysis from social data, and utilizing automated optimization algorithms to further enhance forecasting accuracy across diverse cryptocurrency markets.

REFERENCES

[1] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System.” 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[2] M. Wątorek, S. Drożdż, J. Kwapień, L. Minati, P. Oświęcimka, and M. Stanuszek, “Multiscale characteristics of the emerging global cryptocurrency market,” Physics Reports, vol. 901, pp. 1–82, Nov. 2020, doi: https://doi.org/10.1016/j.physrep.2020.10.005.

[3] A. Bellotti, D. Brigo, P. Gambetti, and F. Vrins, “Forecasting recovery rates on non-performing loans with machine learning,” *Int. J. Forecast.*, vol. 37, no. 1, pp. 428–444, 2021, doi: https://doi.org/10.1016/j.ijforecast.2020.06.009.

[4] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. OTexts, 2021. [Online]. Available: https://otexts.com/fpp3/

[5] M. T. Ismail and R. S. Al-Gounmeein, “Overview of Long Memory for Economic and Financial Time Series Dataset and Related Time Series Models: A Review Study.,” *IAENG Int. J. Appl. Math.*, vol. 52, no. 2, 2022, [Online]. Available: https://www.iaeng.org/IJAM/issues_v52/issue_2/IJAM_52_2_01.pdf

[6] G. G, H. M, and A. K, "Cryptocurrency Price Prediction using Machine Learning," *IJARCCE*, vol. 12, Apr. 2023, doi: https://doi.org/10.17148/IJARCCE.2023.124140.
[7] G. Sonkavde, D. S. Dharrao, A. M. Bongale, S. T. Deokate, D. Doreswamy, and S. K. Bhat, "Forecasting stock market prices using machine learning and deep learning models: A systematic review, performance analysis and discussion of implications," *Int. J. Financ. Stud.*, vol. 11, no. 3, p. 94, 2023, doi: https://doi.org/10.3390/ijfs11030094.
[8] C. Zhang, N. N. A. Sjarif, and R. Ibrahim, "Deep learning models for price forecasting of financial time series: A review of recent advancements: 2020–2022," *WIREs Data Min. Knowl. Discov.*, vol. 14, no. 1, p. e1519, 2024, doi: https://doi.org/10.1002/widm.1519.
[9] J J. Chen, "Analysis of Bitcoin Price Prediction Using Machine Learning," Journal of Risk and Financial Management, vol. 16, no. 1, p. 51, Jan. 2023, doi: https://doi.org/10.3390/jrfm16010051.
[10] M. S. Islam *et al.*, "Machine Learning-Based Cryptocurrency Prediction: Enhancing Market Forecasting with Advanced Predictive Models," *Journal of Ecohumanism*, vol. 4, no. 2, Mar. 2025, doi: https://doi.org/10.62754/joe.v4i2.6663.
[11] M. Z. Islam *et al.*, "Cryptocurrency price forecasting using machine learning: Building intelligent financial prediction models," *ArXiv Prepr. ArXiv250801419*, 2025, doi: https://doi.org/10.48550/arXiv.2508.01419.
[12] S. Siami-Namini, N. Tavakoli, and A. S. Namin, "The performance of LSTM and BiLSTM in forecasting time series," in *Proceedings of the IEEE International Conference on Big Data (IEEE Big Data)*, 2019, pp. 3285–3292. doi: https://doi.org/10.1109/BigData47090.2019.9005997.
[13] F. A. Gers, J. Schmidhuber, and F. Cummins, "Learning to forget: Continual prediction with LSTM," *Neural Comput.*, vol. 12, no. 10, pp. 2451–2471, 2000, doi: https://doi.org/10.1162/089976600300015015.
[14] Q. Guo and others, "MRC-LSTM: A hybrid CNN-LSTM approach for Bitcoin prediction," *ArXiv Prepr. ArXiv210500707*, 2021, doi: https://doi.org/10.48550/arXiv.2105.00707.
[15] X. Li, Q. Liu, and Y. Wu, "Prediction on blockchain virtual currency transaction under long short-term memory model and deep belief network," *Appl. Soft Comput.*, vol. 116, p. 108349, 2022, doi: https://doi.org/10.1016/j.asoc.2021.108349.
[16] H. Song and H. Choi, "Forecasting Stock Market Indices Using the Recurrent Neural Network Based Hybrid Models: CNN-LSTM, GRU-CNN, and Ensemble Models," *Applied Sciences*, vol. 13, no. 7, p. 4644, Apr. 2023, doi: https://doi.org/10.3390/app13074644.
[17] A. Dutta, S. Kumar, and M. Basu, "GRU approach to Bitcoin prediction," *ArXiv Prepr. ArXiv191211166*, 2019, doi: https://doi.org/10.48550/arXiv.1912.11166.
[18] I. Sibel Kervancı and F. Akay, "LSTM Hyperparameters optimization with Hparam parameters for Bitcoin Price Prediction," *Sakarya University Journal of Computer and Information Sciences*, Jan. 2023, doi: https://doi.org/10.35377/saucis.
[19] J. Wang, K. Feng, and G. Qiao, "A hybrid deep learning model for Bitcoin price prediction: data decomposition and feature selection," *Applied economics*, pp. 1–16, Nov. 2023, doi: https://doi.org/10.1080/00036846.2023.2276093.
[20] L. Liao, H. Li, W. Shang, and L. Ma, "An Empirical Study of the Impact of Hyperparameter Tuning and Model Optimization on the Performance Properties of Deep Neural Networks," *ACM Transactions on Software Engineering and Methodology*, vol. 31, no. 3, pp. 1–40, Jul. 2022, doi: https://doi.org/10.1145/3506695.
[21] D. Vidyabharathi and V. Mohanraj, "Hyperparameter Tuning for Deep Neural Networks Based Optimization Algorithm," *Intelligent Automation & Soft Computing*, vol. 0, no. 0, pp. 1–15, 2023, doi: https://doi.org/10.32604/iasc.2023.032255.
[22] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*, 3rd ed. O'Reilly Media, 2022. [Online]. Available: https://www.oreilly.com/library/view/hands-on-machine-learning/9781098125967/
[23] P. Seabe, C. Moutsinga, and E. Pindza, "Forecasting cryptocurrency prices using LSTM and GRU," *Fractal Fract.*, vol. 7, no. 2, p. 203, 2023, doi: https://doi.org/10.3390/fractalfract7020203.
[24] D. S. K. Karunasingha, "Root mean square error or mean absolute error? Use their ratio as well," *Inf. Sci.*, vol. 585, pp. 609–629, 2022. https://doi.org/10.1016/j.ins.2021.11.036
[25] T. O. Hodson, "Root mean square error (RMSE) or mean absolute error (MAE): When to use them or not," *Geosci. Model Dev. Discuss.*, vol. 2022, pp. 1–10, 2022. https://gmd.copernicus.org/articles/15/5481/2022/
[26] S. Román, S. Z. Collazos, and J. C. Chavez, "Model for Reducing Mean Absolute Percentage Error through Smoothing and Time Series Forecasting In a Tourism SME: A Case Study," *J. Mach. Intell. Data Sci. JMIDS*, vol. 5, 2024. doi: https://doi.org/10.11159/jmids.2024.012